

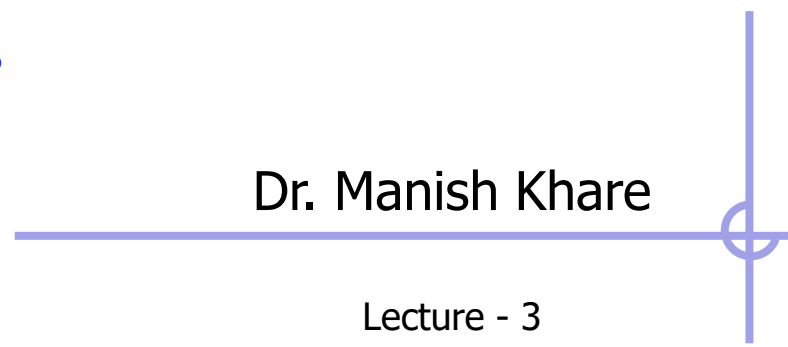
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture - 3



Asymptotic Notations

➤ Following are the commonly used asymptotic notations to calculate the running time complexity of an algorithm.

- O Notation
- Ω Notation
- θ Notation

Big O notation

- What do we mean when we measure runtime?
 - We probably care about wall time: how long does it take to solve the problem, in seconds or minutes or hours?
- This is heavily dependent on the programming language, architecture, etc.
- These things are very important, but are not the point of this class.
- We want a way to talk about the running time of an algorithm, independent of these considerations.

- Focus on how the runtime **scales** with n (the input size).
- Informally

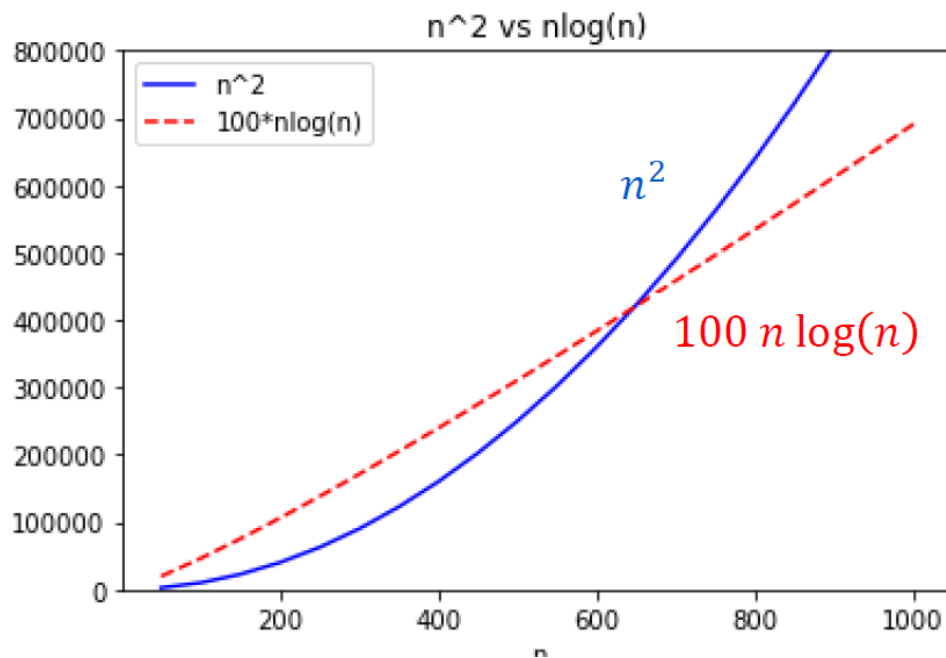
Number of operations	Asymptotic Running Time
$\frac{1}{10} n^2 + 100$	$O(n^2)$
$0.063 n^2 - .5 n + 12.7$	$O(n^2)$
$100 n^{1.5} - 10^{10000} \sqrt{n}$	$O(n^{1.5})$
$11 n \log(n) + 1$	$O(n \log(n))$

(Only pay attention to the largest function of n that appears.)

We say this algorithm is “asymptotically faster” than the others.

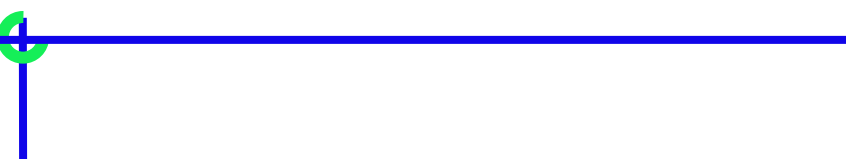
➤ So $100n\log(n)$ operations is “better” than n^2 operations?

But when
 $n=200$,
that's not
true at all!



Yeah, but it's
true once n
is at least
700 or so.



- 
- One algorithm is “faster” than another if its runtime scales better with the size of the input.

Pros:

- Abstracts away from hardware- and language-specific issues.
- Makes algorithm analysis much more tractable.

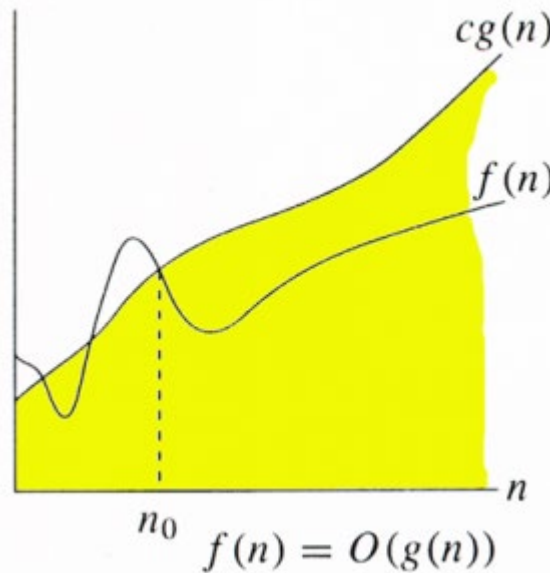
Cons:

- Only makes sense if n is large (compared to the constant factors).

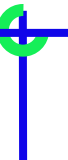
$1000000000 n$
is “better” than n^2 ?!?!

Big Oh Notation, O

- The notation $O(n)$ is the formal way to express the upper bound of an algorithm's running time. It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.



$O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$



➤ $O(\dots)$ means an upper bound
 ↙ pronounced “big-oh of ...” or sometimes “oh of ...”

- Let $T(n)$, $g(n)$ be functions of positive integers.
 - Think of $T(n)$ as a runtime: positive and increasing in n .
- We say “ $T(n)$ is $O(g(n))$ ” if $T(n)$ grows no faster than $g(n)$ as n gets large.
- Formally,

$$T(n) = O(g(n))$$

$$\Leftrightarrow$$

$$\exists c, n_0 > 0 \text{ s.t. } \forall n \geq n_0,$$

$$0 \leq T(n) \leq c \cdot g(n)$$

Example

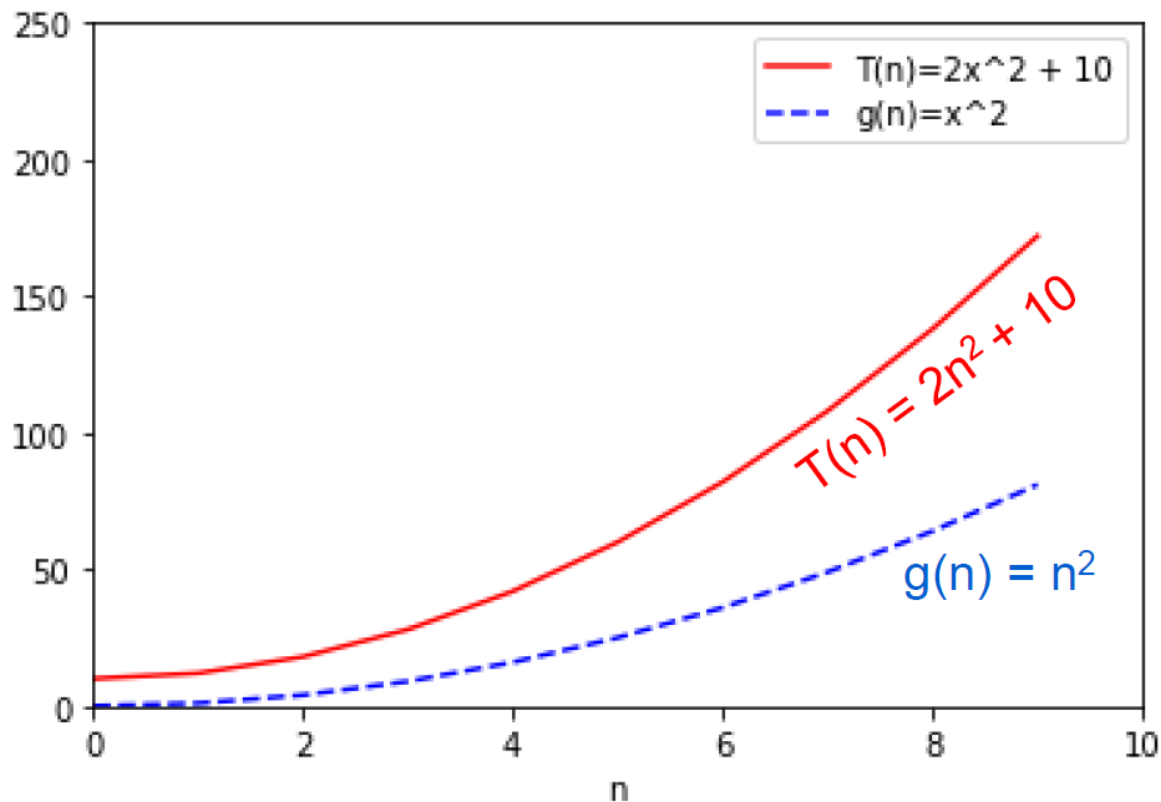
$$2n^2 + 10 = O(n^2)$$

$$T(n) = O(g(n))$$

$$\Leftrightarrow$$

$$\exists c, n_0 > 0 \text{ s.t. } \forall n \geq n_0,$$

$$0 \leq T(n) \leq c \cdot g(n)$$



Example

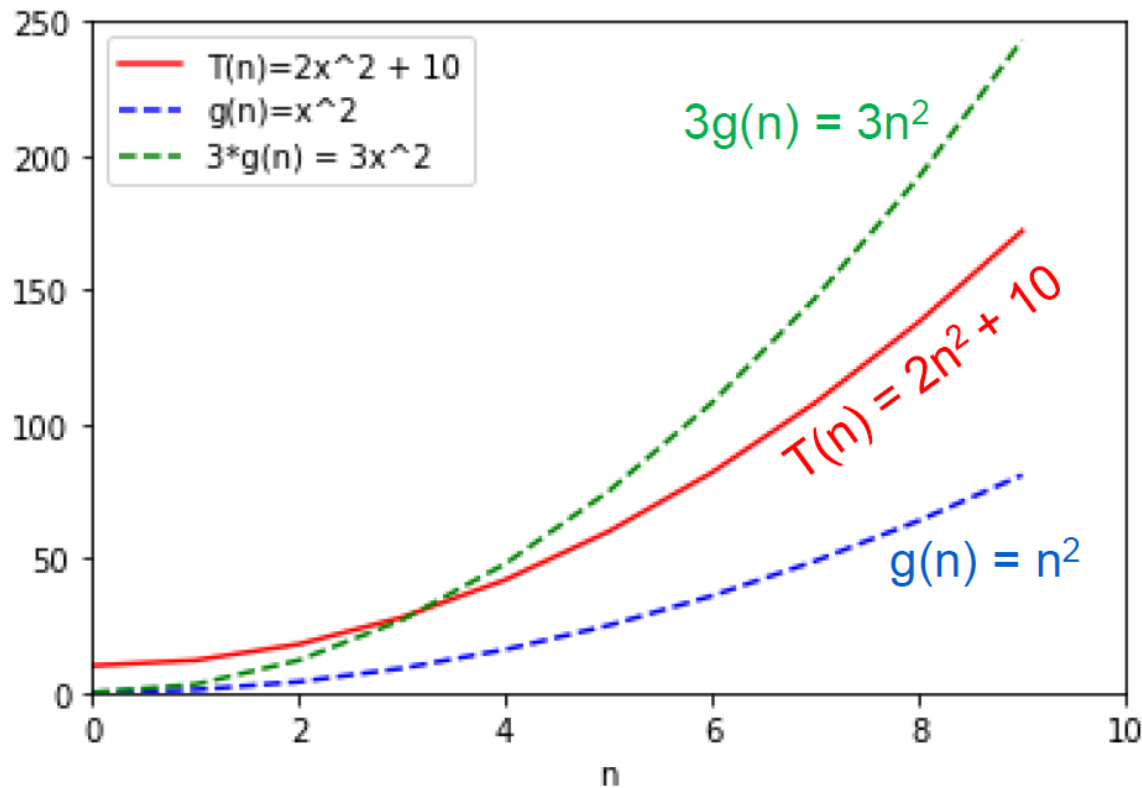
$$2n^2 + 10 = O(n^2)$$

$$T(n) = O(g(n))$$

$$\Leftrightarrow$$

$$\exists c, n_0 > 0 \text{ s.t. } \forall n \geq n_0,$$

$$0 \leq T(n) \leq c \cdot g(n)$$



Example

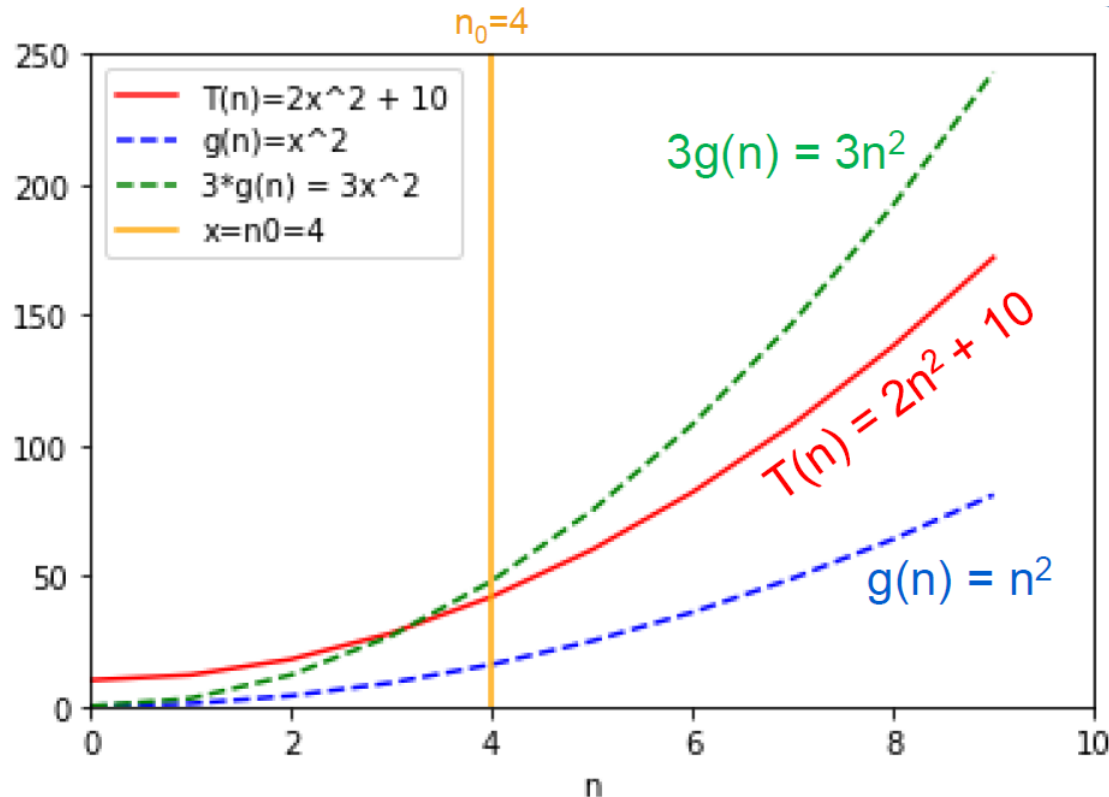
$$2n^2 + 10 = O(n^2)$$

$$T(n) = O(g(n))$$

$$\Leftrightarrow$$

$$\exists c, n_0 > 0 \text{ s.t. } \forall n \geq n_0,$$

$$0 \leq T(n) \leq c \cdot g(n)$$



Example

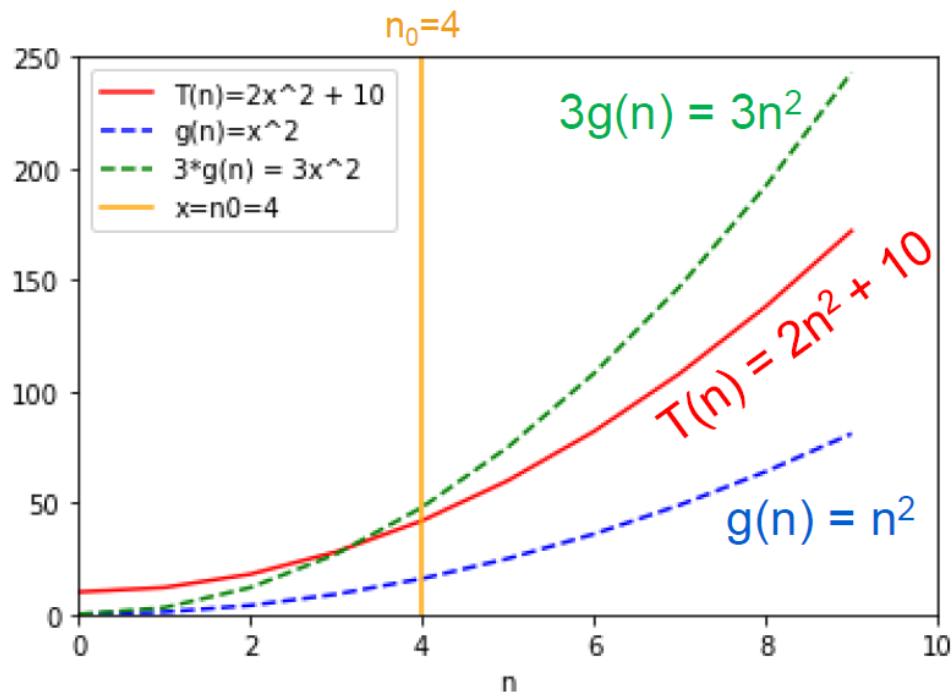
$$2n^2 + 10 = O(n^2)$$

$$T(n) = O(g(n))$$

$$\Leftrightarrow$$

$$\exists c, n_0 > 0 \text{ s.t. } \forall n \geq n_0,$$

$$0 \leq T(n) \leq c \cdot g(n)$$



Formally:

- Choose $c = 3$
- Choose $n_0 = 4$
- Then:

$$\forall n \geq 4,$$

$$0 \leq 2n^2 + 10 \leq 3 \cdot n^2$$

Example

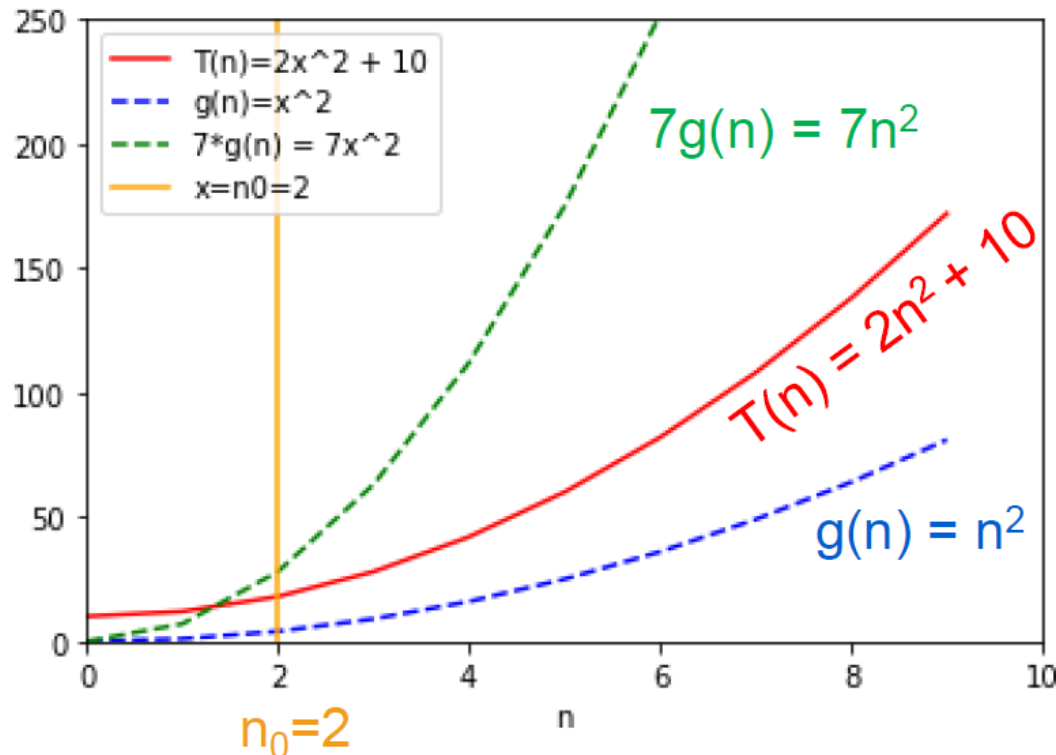
$$2n^2 + 10 = O(n^2)$$

$$T(n) = O(g(n))$$

$\Leftrightarrow$

$\exists c, n_0 > 0$ s.t. $\forall n \geq n_0$,

$$0 \leq T(n) \leq c \cdot g(n)$$



Formally:

- Choose $c = 7$
- Choose $n_0 = 2$
- Then:

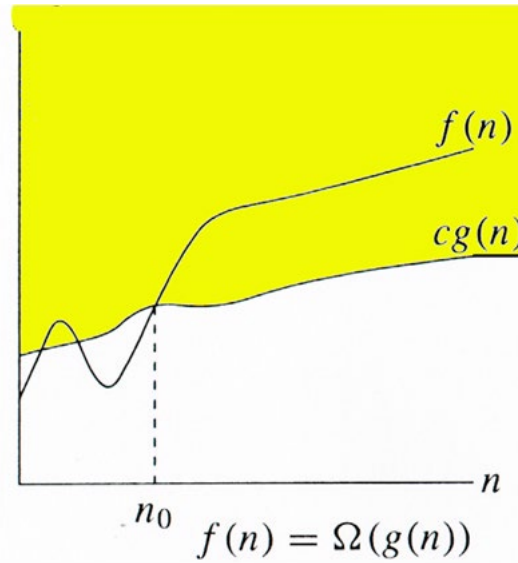
$$\forall n \geq 2,$$

$$0 \leq 2n^2 + 10 \leq 7 \cdot n^2$$

There is not a
“unique” choice
of c and n_0

Omega Notation, Ω

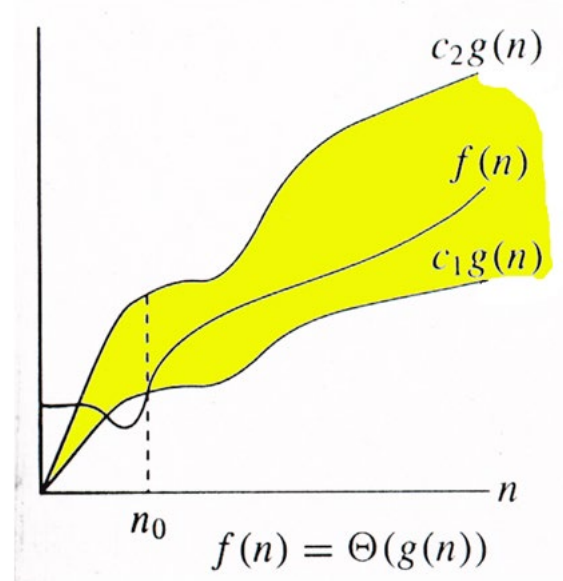
- The notation $\Omega(n)$ is the formal way to express the lower bound of an algorithm's running time. It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.



$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}.$

Theta Notation, Θ

➤ The notation $\Theta(n)$ is the formal way to express both the lower bound and the upper bound of an algorithm's running time. It is represented as follows.



$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}$

Common Asymptotic Notations

constant	—	$O(1)$
logarithmic	—	$O(\log n)$
linear	—	$O(n)$
$n \log n$	—	$O(n \log n)$
quadratic	—	$O(n^2)$
cubic	—	$O(n^3)$
polynomial	—	$n^{O(1)}$
exponential	—	$2^{O(n)}$

Algorithms

- 
- Greedy Algorithms
 - Divide and Conquer Algorithm
 - Dynamic Programming Algorithm
 - Backtracking
 - Branch and Bound

Greedy Algorithms

- An algorithm is designed to achieve optimum solution for a given problem. In greedy algorithm approach, decisions are made from the given solution domain. As being greedy, the closest solution that seems to provide an optimum solution is chosen.
- Greedy algorithms try to find a localized optimum solution, which may eventually lead to globally optimized solutions. However, generally greedy algorithms do not provide globally optimized solutions.

Greedy Algorithms – Example (Counting Coins)

- This problem is to count to a desired value by choosing the least possible coins and the greedy approach forces the algorithm to pick the largest possible coin. If we are provided coins of ₹ 1, 2, 5 and 10 and we are asked to count ₹ 18 then the greedy procedure will be –
 - 1 – Select one ₹ 10 coin, the remaining count is 8
 - 2 – Then select one ₹ 5 coin, the remaining count is 3
 - 3 – Then select one ₹ 2 coin, the remaining count is 1
 - 4 – And finally, the selection of one ₹ 1 coins solves the problem
- Though, it seems to be working fine, for this count we need to pick only 4 coins. But if we slightly change the problem then the same approach may not be able to produce the same optimum result.

Greedy Algorithms – Example (Counting Coins)

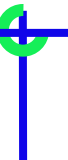
- For the currency system, where we have coins of 1, 7, 10 value, counting coins for value 18 will be absolutely optimum but for count like 15, it may use more coins than necessary. For example, the greedy approach will use $10 + 1 + 1 + 1 + 1 + 1$, total 6 coins. Whereas the same problem could be solved by using only 3 coins ($7 + 7 + 1$).
- Hence, we may conclude that the greedy approach picks an immediate optimized solution and may fail where global optimization is a major concern.

Greedy Algorithms – Example

- Most networking algorithms use the greedy approach. Here is a list of few of them –
 - Travelling Salesman Problem
 - Prim's Minimal Spanning Tree Algorithm
 - Kruskal's Minimal Spanning Tree Algorithm
 - Dijkstra's Minimal Spanning Tree Algorithm
 - Graph - Map Coloring
 - Graph - Vertex Cover
 - Knapsack Problem
 - Job Scheduling Problem
- There are lots of similar problems that uses the greedy approach to find an optimum solution

Divide and Conquer Algorithm

- In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently.
- When we keep on dividing the subproblems into even smaller sub-problems, we may eventually reach a stage where no more division is possible.
- Those "atomic" smallest possible sub-problem (fractions) are solved.
- The solution of all sub-problems is finally merged in order to obtain the solution of an original problem



➤ Divide/Break

- This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem. This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible. At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.

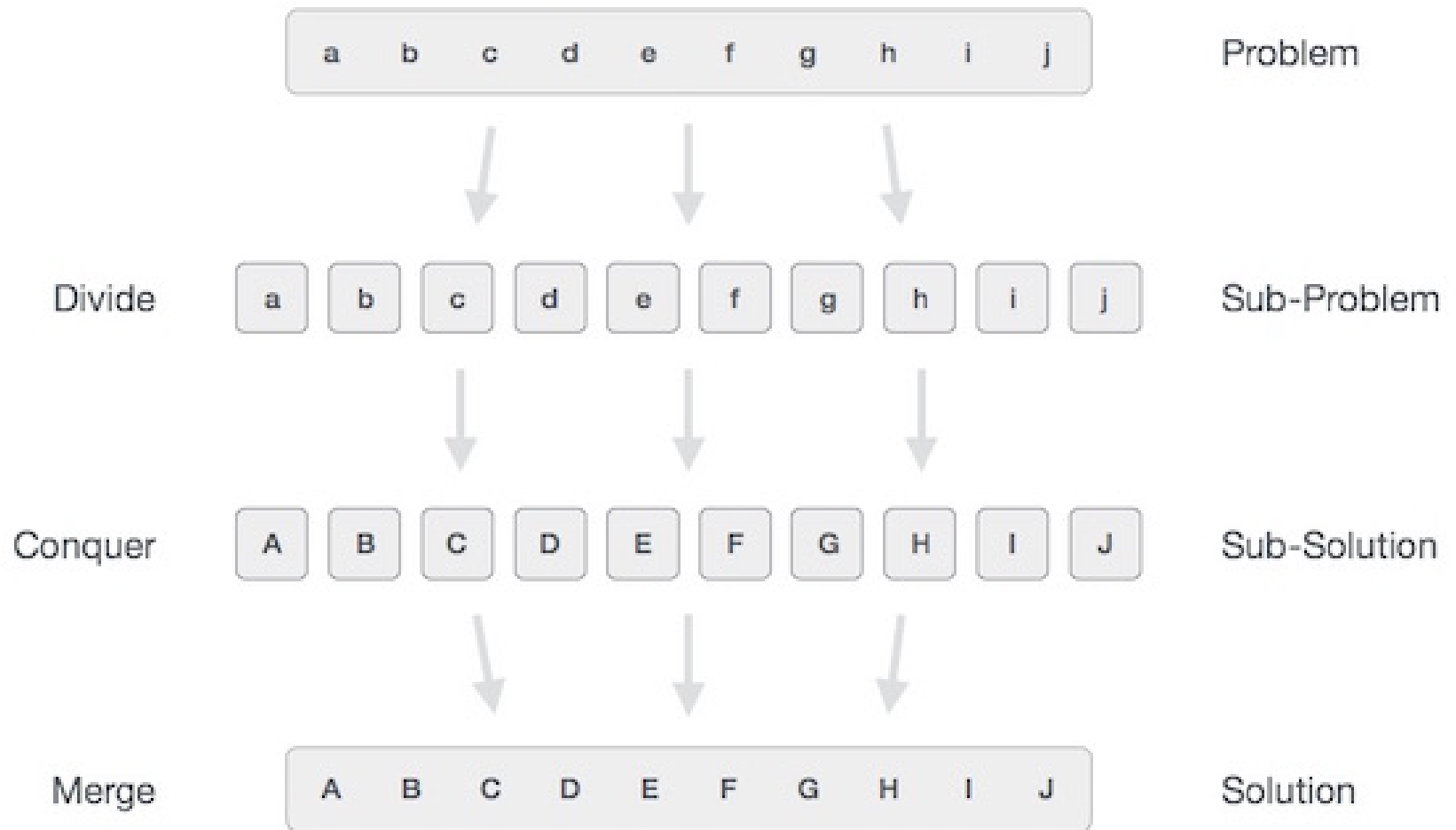
➤ Conquer/Solve

- This step receives a lot of smaller sub-problems to be solved. Generally, at this level, the problems are considered 'solved' on their own.

➤ Merge/Combine

- When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem. This algorithmic approach works recursively and conquer & merge steps work so close that they appear as one.

Divide and Conquer Algorithm



Divide and Conquer Algorithm - Examples

- The following computer algorithms are based on **divide-and-conquer** programming approach –
 - Merge Sort
 - Quick Sort
 - Binary Search
 - Strassen's Matrix Multiplication
 - Closest pair (points)

- There are various ways available to solve any computer problem, but the mentioned are a good example of divide and conquer approach.

Dynamic Programming

- Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems. But unlike, divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.
- Dynamic programming is used where we have problems, which can be divided into similar sub-problems, so that their results can be re-used.
- Mostly, these algorithms are used for optimization. Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems.
- The solutions of sub-problems are combined in order to achieve the best solution.

Dynamic Programming

➤ we can say that –

- The problem should be able to be divided into smaller overlapping sub-problem.
- An optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- Dynamic algorithms use memorization.

Dynamic Programming - Example

- The following computer problems can be solved using dynamic programming approach –
 - Fibonacci number series
 - Knapsack problem
 - Tower of Hanoi
 - All pair shortest path by Floyd-Warshall
 - Shortest path by Dijkstra
 - Project scheduling
- Dynamic programming can be used in both top-down and bottom-up manner. And of course, most of the times, referring to the previous solution output is cheaper than recomputing in terms of CPU cycles.

Comparison

- In contrast to greedy algorithms, where local optimization is addressed, dynamic algorithms are motivated for an overall optimization of the problem.
- In contrast to divide and conquer algorithms, where solutions are combined to achieve an overall solution, dynamic algorithms use the output of a smaller sub-problem and then try to optimize a bigger sub-problem. Dynamic algorithms use memorization to remember the output of already solved sub-problems



Data structures basic concepts

Data Definition & Data Objects

- Data Definition defines a particular data with the following characteristics.
 - **Atomic** – Definition should define a single concept.
 - **Traceable** – Definition should be able to be mapped to some data element.
 - **Accurate** – Definition should be unambiguous.
 - **Clear and Concise** – Definition should be understandable.
- Data Object represents an object having a data.

Data Type

- 
- Data type is a way to classify various types of data such as integer, string, etc. which determines the values that can be used with the corresponding type of data, the type of operations that can be performed on the corresponding type of data. There are two data types –
 - Built-in Data Type
 - Derived Data Type

Build-in Data Type

➤ Those data types for which a language has built-in support are known as Built-in Data types. For example, most of the languages provide the following built-in data types.

- Integers
- Boolean (true, false)
- Floating (Decimal numbers)
- Character and Strings

Derived Data Type

- Those data types which are implementation independent as they can be implemented in one or the other way are known as derived data types. These data types are normally built by the combination of primary or built-in data types and associated operations on them.
- For example –
 - List
 - Array
 - Stack
 - Queue